

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO



FACULTAD DE CIENCIAS DE LA COMPUTACIÓN CARRERA DE INGENIERÍA EN SOFTWARE

TEMA:

PRÁCTICA EXPERIMENTAL – UNIDAD IV

Validación, Gestión de Requisitos y Herramientas CASE

ASIGNATURA:

INGENIERÍA DE REQUERIMIENTOS

DOCENTE:

ING. GUERRERO ULLOA GLEISTON CICERÓN

INTEGRANTES:

CEDEÑO AVILA WINSTON DAMIAN, CI: 0942833492, wcedenoa2@uteq.edu.ec

CORDOVA CARREÑO MAYRA LUCILA, CI: 0750286775, mcordovac2@uteq.edu.ec

MENDOZA PÁRRAGA ANDY JOHEL, CI: 1251401590, amendozap9@uteq.edu.ec, Titular PFC

EQUIPO:

B

CURSO/PARALELO:

CUARTO SEMESTRE PARALELO “B”

SISTEMA PFC:

MundiPets (Andy Mendoza – Titular PFC)

REPOSITORIO GITHUB:

<https://github.com/andymendozap03/ISR401-PFC-ERS-EQUIPOB.git>

QUEVEDO – LOS RÍOS – ECUADOR
2026 – 2027 PPA

Índice

1.	Resumen y objetivos	3
1.1.	Objetivo general	3
1.2.	Objetivos específicos	3
2.	Fundamento teórico	4
2.1.	Marco normativo	4
2.1.1.	¿Qué regula la validación y gestión de requisitos?	4
2.1.2.	IEEE/ISO/IEC 29148:2018 – Proceso de validación de requisitos de software	4
2.1.3.	IEEE/ISO/IEC 15288:2023 y 12207:2017 – Procesos de control de cambios	4
2.1.4.	ISO/IEC 25010:2023 – Modelo de calidad aplicado al ERS	5
2.1.5.	SWEBOK v4.0 y Pohl (2025) Parte V	5
2.1.6.	El CCB según PMBOK 7. ^a edición	5
2.2.	Validación de requisitos	5
2.2.1.	Verificación vs. validación: diferencias conceptuales	6
2.2.2.	Principios de validación de requisitos	6
2.2.3.	Técnicas de validación	6
2.2.4.	Inspección Fagan: origen, fases y roles	6
2.2.5.	Métricas de inspección	7
2.2.6.	Caso de estudio o ejemplo aplicado	7
2.3.	Gestión de requisitos	7
2.3.1.	Gestión de la configuración del ERS	8
2.3.2.	Línea base (baseline) y control de versiones	8
2.3.3.	Atributos de los requisitos	8
2.3.4.	Trazabilidad: pre-traceability y post-traceability	8
2.3.5.	Matriz de trazabilidad	9
2.3.6.	Proceso de cambio: RFC, análisis de impacto, CCB, implementación y cierre	9
2.3.7.	Métricas de volatilidad de requisitos	9
2.4.	Herramientas CASE para IR	10
2.4.1.	Clasificación de herramientas CASE	10
2.4.2.	Funcionalidades esperables en gestión de requisitos	10
2.4.3.	Criterios de selección de una herramienta CASE	11
2.4.4.	Limitaciones y costos	11
2.5.	IR en procesos ágiles	11
2.5.1.	Product backlog, historias de usuario y criterios de aceptación	11
2.5.2.	Grooming y Definition of Ready / Definition of Done	11
2.5.3.	BDD y formato Gherkin	12
2.5.4.	Trazabilidad en entornos ágiles	12
2.5.5.	IR documental frente a IR ágil: ¿cuándo conviene cada enfoque?	12
3.	Inspección formal del ERS (método Fagan)	13
3.1.	Planificación y roles	13
3.2.	Preparación individual	13
3.3.	Acta de la reunión de inspección	13
3.4.	Métricas de inspección	13
4.	Correcciones aplicadas	14
5.	Gestión del cambio y Change Control Board (CCB)	15
5.1.	RFC-01 – Cambio de alcance (nuevo requisito)	15
5.2.	RFC-02 – Cambio de calidad (modificación de un RNF)	15
5.3.	RFC-03 – Cambio de restricción o normativa	15
5.4.	Acta del CCB	15
5.5.	Versión resultante del ERS	15
6.	Trazabilidad en herramienta CASE	16
6.1.	Estructura del tablero	16
6.2.	Campos personalizados	16

6.3.	Matriz de trazabilidad	16
6.4.	Evidencias del tablero	16
7.	Línea base del ERS con Git	17
7.1.	Repositorio y estructura	17
7.2.	Commits semánticos	17
7.3.	Tag anotado de línea base	17
7.4.	CHANGELOG.md	17
7.5.	Historial de commits (evidencia)	17
8.	Comparativa de herramientas CASE	18
9.	IR tradicional frente a IR ágil	19
10.	Conclusiones críticas	20
11.	Distribución del trabajo	21
12.	Referencias	22
Anexo A — Lista de verificación de inspección (checklists individuales)		24
Anexo B — Registro de defectos		25
Anexo C — Formularios RFC y acta del CCB		26
Anexo D — Exportación CSV del backlog		27
Anexo E — Capturas del tablero CASE y del repositorio Git		28
Anexo F — Referencias IEEE y declaración de uso de Inteligencia Artificial		29

1. Resumen y objetivos

1.1. Objetivo general

Aplicar técnicas formales de validación de requisitos, gestionar cambios mediante un Change Control Board (CCB) simulado, implementar la trazabilidad del ERS en una herramienta CASE, establecer una línea base con Git y comparar metodológicamente la Ingeniería de Requisitos (IR) tradicional frente a la IR ágil, sobre el ERS real del Proyecto Fin de Curso MundiPets.

1.2. Objetivos específicos

1. Ejecutar una inspección formal tipo Fagan sobre el ERS del PFC, con roles diferenciados, lista de verificación y registro de defectos.
2. Calcular e interpretar métricas de inspección (densidad de defectos, distribución por tipo y severidad, tasa de corrección).
3. Tramitar tres solicitudes formales de cambio (RFC) con análisis de impacto sustentado en la matriz de trazabilidad.
4. Deliberar y decidir en un CCB simulado, dejando acta motivada y versión actualizada del ERS.
5. Construir la trazabilidad bidireccional del ERS en una herramienta CASE de gestión (Jira o Trello) con campos personalizados y enlaces verificables.
6. Establecer y publicar la línea base del ERS con control de versiones (commit semántico, tag anotado y `CHANGELOG.md`).
7. Comparar herramientas CASE de IR y contrastar la IR tradicional frente a la IR ágil, cerrando con conclusiones críticas propias.

2. Fundamento teórico

2.1. Marco normativo

La ingeniería de requisitos se apoya en estándares, cuerpos de conocimiento y marcos metodológicos que establecen principios para especificar, validar, mantener y controlar los requisitos durante el ciclo de vida de un sistema. Entre los principales referentes se encuentran ISO/IEC/IEEE 29148:2018 para los procesos de ingeniería de requisitos, ISO/IEC/IEEE 15288:2023 e ISO/IEC/IEEE 12207:2017 para los procesos del ciclo de vida, ISO/IEC 25010:2023 para las características de calidad, así como SWEBOK v4.0 y los fundamentos contemporáneos de ingeniería de requisitos desarrollados por Pohl. Estos referentes permiten abordar la calidad de los requisitos no únicamente desde su redacción, sino también desde su verificabilidad, trazabilidad y evolución durante el desarrollo.

La relevancia de estos mecanismos continúa siendo respaldada por investigaciones recientes. Un estudio de mapeo publicado en *Requirements Engineering* señala que la calidad de los requisitos constituye un área multidimensional cuya evaluación involucra diferentes propiedades, defectos y mecanismos de aseguramiento [1]. Esta perspectiva justifica la utilización conjunta de estándares y prácticas de ingeniería para establecer criterios consistentes de calidad y control de los requisitos.

2.1.1. ¿Qué regula la validación y gestión de requisitos?

La validación y la gestión de requisitos comprenden actividades destinadas a asegurar que los requisitos representen adecuadamente las necesidades de los interesados y permanezcan controlados durante la evolución del proyecto. La validación se orienta a determinar si los requisitos especificados corresponden con las necesidades y objetivos que el sistema debe satisfacer, mientras que la gestión considera aspectos como los atributos de los requisitos, su trazabilidad, priorización, versionado y tratamiento de cambios.

Estas actividades no deben entenderse como procesos aislados. La modificación de un requisito puede afectar otros requisitos y artefactos asociados, por lo que resulta necesario mantener relaciones que permitan conocer su procedencia y las consecuencias de su evolución. Investigaciones recientes sobre ingeniería de requisitos identifican precisamente la validación y la gestión como actividades fundamentales: la primera busca comprobar la calidad y adecuación de la información obtenida, mientras que la segunda permite seguir los cambios producidos en los requisitos y mantenerlos acordes con las necesidades de los stakeholders [2].

2.1.2. IEEE/ISO/IEC 29148:2018 – Proceso de validación de requisitos de software

ISO/IEC/IEEE 29148:2018 constituye uno de los principales referentes normativos para la ingeniería de requisitos de sistemas y software. El estándar establece procesos y disposiciones relacionadas con la identificación, formulación, análisis y gestión de requisitos a lo largo del ciclo de vida [3]. Dentro de este contexto, la validación permite determinar si los requisitos y la especificación resultante representan adecuadamente las necesidades de los interesados y los objetivos establecidos para el sistema.

La calidad de un requisito implica que pueda comprenderse y utilizarse posteriormente para actividades de diseño, implementación y comprobación. Por esta razón, la especificación debe procurar características que reduzcan ambigüedades, inconsistencias y dificultades de interpretación. La investigación empírica reciente confirma que la calidad de requisitos continúa siendo un problema relevante en ingeniería de software y que sus atributos y defectos requieren métodos sistemáticos de evaluación [1].

2.1.3. IEEE/ISO/IEC 15288:2023 y 12207:2017 – Procesos de control de cambios

ISO/IEC/IEEE 15288:2023 proporciona un marco de procesos para el ciclo de vida de sistemas, mientras que ISO/IEC/IEEE 12207:2017 establece procesos aplicables al ciclo de vida del software [4, 5]. Dentro de este marco, los requisitos no se consideran elementos estáticos: pueden evolucionar como consecuencia de nuevas necesidades, restricciones, decisiones técnicas o cambios en el contexto del sistema.

Por ello, el control de cambios debe permitir identificar una modificación, analizar sus posibles consecuencias y mantener consistentes los elementos afectados. En ingeniería de requisitos esto adquiere especial importancia porque una modificación puede repercutir en requisitos relacionados, diseño, implementación y pruebas. Consecuentemente, la trazabilidad constituye un mecanismo de apoyo fundamental para determinar dichas relaciones.

Esta necesidad está respaldada por estudios contemporáneos sobre trazabilidad, que señalan que mantener los requisitos actualizados y comprender su contexto resulta especialmente importante conforme aumenta el tamaño y complejidad del proyecto. La trazabilidad permite recuperar información sobre el origen, las modificaciones y las personas involucradas en un requisito, proporcionando la información necesaria para gestionar su evolución [6].

2.1.4. ISO/IEC 25010:2023 – Modelo de calidad aplicado al ERS

ISO/IEC 25010:2023 establece un modelo de calidad que permite caracterizar y evaluar la calidad de productos software y sistemas [7]. Aunque el estándar no sustituye a ISO/IEC/IEEE 29148 como marco específico de ingeniería de requisitos, sus características de calidad pueden utilizarse para orientar la definición de requisitos no funcionales y atributos de calidad dentro de una Especificación de Requisitos de Software [3].

Esto significa que las características de calidad relevantes para el producto deben transformarse en requisitos suficientemente claros y evaluables. De esta forma, el ERS actúa como vínculo entre las necesidades de calidad identificadas y las posteriores actividades de diseño, implementación y pruebas.

La importancia de definir correctamente estos atributos está respaldada por la investigación reciente sobre calidad de requisitos, que encuentra una diversidad considerable de atributos y problemas utilizados para estudiar empíricamente la calidad de los requisitos, lo que evidencia la necesidad de utilizar marcos estructurados para definir y evaluar sus propiedades [1].

2.1.5. SWEBOK v4.0 y Pohl (2025) Parte V

El *Guide to the Software Engineering Body of Knowledge (SWEBOK)* v4.0, publicado por IEEE Computer Society en 2024, consolida el conocimiento generalmente aceptado dentro de la ingeniería de software. Su área dedicada a requisitos contempla fundamentos, elicitación, análisis y otras actividades necesarias para transformar necesidades de los interesados en requisitos que puedan ser utilizados durante el desarrollo del software [8, 9]. La versión 4.0 actualiza además el cuerpo de conocimiento para incorporar prácticas contemporáneas de ingeniería de software.

Complementariamente, la segunda edición de *Requirements Engineering: Fundamentals, Principles, and Techniques*, publicada por Springer en 2025, presenta un tratamiento sistemático de la ingeniería de requisitos. Particularmente, la Parte V, denominada *Cross-Sectional Activities*, desarrolla cuatro actividades de validación y tres actividades relacionadas con la gestión de requisitos. Este planteamiento permite comprender ambas actividades como elementos transversales que acompañan la evolución de los requisitos y no únicamente como tareas realizadas al finalizar su especificación [10].

La combinación de SWEBOK v4.0 y Pohl proporciona, por tanto, una perspectiva complementaria al marco normativo: mientras SWEBOK sintetiza el cuerpo de conocimiento aceptado de la disciplina, Pohl profundiza en principios, procedimientos y técnicas específicas para validar y gestionar requisitos.

2.1.6. El CCB según PMBOK 7.ª edición

El control formal de cambios requiere establecer responsabilidades para evaluar las modificaciones propuestas antes de incorporarlas a una línea base. En este contexto se utiliza el Change Control Board (CCB) o Comité de Control de Cambios, entendido como un grupo formalmente establecido al que se asigna autoridad para evaluar determinadas solicitudes de cambio conforme al procedimiento definido para el proyecto [11].

Aplicado a la ingeniería de requisitos, una solicitud de modificación debe registrarse y someterse a análisis antes de modificar directamente el ERS o su línea base. El análisis de impacto permite determinar qué requisitos, artefactos, componentes, costos, plazos o pruebas podrían verse afectados. Con esa información puede adoptarse una decisión sobre la aprobación, rechazo o aplazamiento del cambio y, cuando corresponda, actualizar los elementos afectados manteniendo su trazabilidad [11].

2.2. Validación de requisitos

La validación de requisitos constituye una actividad esencial de la ingeniería de requisitos orientada a determinar si la especificación representa adecuadamente las necesidades, expectativas y objetivos de los interesados. Su importancia radica en detectar problemas antes de que estos se propaguen hacia etapas posteriores del desarrollo, donde su corrección puede implicar modificaciones en el diseño, implementación y pruebas del sistema.

La literatura reciente continúa considerando la validación como una de las actividades fundamentales de la ingeniería de requisitos, señalando que busca garantizar que la información recopilada se encuentre correctamente organizada para satisfacer los objetivos del sistema, evaluando aspectos como exactitud, completitud y corrección de los documentos o modelos de requisitos [2].

Esta actividad se relaciona directamente con la calidad del ERS. Entre los atributos estudiados con mayor frecuencia en la calidad de requisitos se identifican la ambigüedad, completitud, consistencia y corrección, características cuya evaluación permite descubrir deficiencias antes de utilizar la especificación como base para las siguientes etapas del desarrollo [1].

2.2.1. Verificación vs. validación: diferencias conceptuales

Aunque los términos verificación y validación suelen aparecer conjuntamente, representan objetivos diferentes dentro del aseguramiento de la calidad. La verificación se concentra principalmente en comprobar si los requisitos han sido especificados de manera adecuada, es decir, si presentan las propiedades necesarias para poder ser comprendidos, analizados y posteriormente comprobados. En este proceso pueden examinarse características como consistencia, completitud, ausencia de ambigüedad y verificabilidad.

La validación, en cambio, se orienta hacia la correspondencia entre los requisitos documentados y las necesidades reales de los stakeholders. Por tanto, un requisito puede estar correctamente redactado desde el punto de vista formal y aun así no representar lo que necesita el usuario. La validación requiere entonces involucrar a los interesados para determinar si lo especificado describe el sistema que realmente se pretende desarrollar.

La diferencia puede sintetizarse en dos preguntas: la verificación analiza si los requisitos están correctamente especificados, mientras que la validación determina si se han especificado los requisitos adecuados. Sin embargo, ambas actividades son complementarias y contribuyen conjuntamente al aseguramiento de la calidad del ERS.

La importancia de la verificabilidad se evidencia en enfoques destinados específicamente a verificar requisitos de rendimiento y generar entornos de prueba a partir de ellos, lo cual muestra la relación existente entre requisitos correctamente formulados y su posterior comprobación [12].

2.2.2. Principios de validación de requisitos

La validación debe realizarse de manera sistemática y considerar tanto las características individuales de cada requisito como la consistencia del conjunto completo. Un requisito aislado puede parecer adecuado, pero generar contradicciones cuando se relaciona con otros elementos del ERS. Por ello, la revisión debe considerar la especificación desde una perspectiva integral.

Entre los aspectos fundamentales se encuentran la corrección, para determinar si el requisito representa una necesidad real; la completitud, para comprobar que la información necesaria haya sido especificada; la consistencia, para identificar contradicciones entre requisitos; y la ausencia de ambigüedad, de modo que un requisito no admita interpretaciones incompatibles.

Estos aspectos están respaldados por el estudio sistemático de Montgomery et al., que identifica doce atributos de calidad estudiados empíricamente y encuentra que ambigüedad, completitud, consistencia y corrección se encuentran entre los más prominentes en la investigación sobre calidad de requisitos [1].

La validación tampoco debe concebirse exclusivamente como una actividad final. Los requisitos evolucionan y pueden ser modificados como resultado de nuevas necesidades o de la detección de inconsistencias. Por ello, resulta conveniente realizar actividades de validación de manera iterativa durante su elaboración y después de cambios relevantes.

2.2.3. Técnicas de validación

La validación de requisitos puede realizarse mediante diferentes técnicas cuya selección depende de las características del proyecto, los artefactos disponibles y la participación de los stakeholders. Entre las alternativas se encuentran las revisiones e inspecciones, el uso de prototipos, la construcción y evaluación de escenarios, la utilización de listas de comprobación y la derivación temprana de casos o criterios de prueba.

Las revisiones permiten que diferentes participantes examinen los requisitos para localizar omisiones, contradicciones, ambigüedades o información incorrecta. Los prototipos permiten representar anticipadamente determinadas características o interacciones del sistema, facilitando que los usuarios evalúen si el comportamiento esperado corresponde con sus necesidades. Los escenarios, por su parte, describen situaciones concretas de utilización y permiten analizar cómo los requisitos se manifiestan durante una interacción determinada.

La investigación reciente demuestra que las prácticas de validación continúan evolucionando según el contexto tecnológico. Un estudio de mapeo sobre ingeniería de requisitos para sistemas basados en inteligencia artificial identificó 53 estudios relacionados con validación de requisitos, encontrando prácticas como entrevistas, cuestionarios y escenarios, además de nuevas propuestas adaptadas a estos sistemas [13]. La utilización conjunta de diferentes técnicas puede resultar especialmente útil, debido a que una sola técnica difícilmente permite detectar todos los tipos de defectos presentes en una especificación.

2.2.4. Inspección Fagan: origen, fases y roles

La Inspección Fagan constituye un método formal y estructurado de revisión introducido originalmente por Michael Fagan para la detección sistemática de defectos en artefactos de software. Aunque su origen es anterior al intervalo bibliográfico establecido para los artículos de este trabajo, continúa siendo utilizado como referente conceptual en investigaciones contemporáneas relacionadas con inspecciones [14].

Aplicada a un ERS, la inspección permite examinar sistemáticamente los requisitos para identificar defectos antes de avanzar hacia etapas posteriores. El proceso puede organizarse en actividades de planificación, preparación individual, reunión de inspección, corrección o retrabajo y seguimiento. Durante estas actividades se distribuyen responsabilidades entre participantes como el autor del artefacto, el moderador o responsable de conducir la inspección, los inspectores o revisores y, cuando se establece como función independiente, el encargado de registrar los defectos encontrados.

La finalidad principal de la inspección no consiste en rediseñar inmediatamente el requisito durante la reunión, sino en identificar y registrar sus defectos para que posteriormente puedan ser corregidos y verificados.

La vigencia de la inspección de requisitos puede observarse en investigaciones recientes que estudian los fallos dependientes producidos durante la inspección de requisitos, y destacan que la especificación suele ser examinada por un equipo de inspectores para detectar defectos, debido a la influencia que su calidad ejerce sobre las fases posteriores y sobre el producto software [14].

2.2.5. Métricas de inspección

Las métricas de inspección permiten evaluar cuantitativamente los resultados de una revisión y proporcionan información para valorar la efectividad del proceso. Entre las medidas que pueden registrarse se encuentran el número total de defectos identificados, su clasificación por tipo o severidad, el tiempo empleado en la inspección, la cantidad de requisitos examinados y la distribución de defectos entre las diferentes partes del ERS.

A partir de estas medidas pueden construirse indicadores como la densidad de defectos, relacionando la cantidad de defectos encontrados con el tamaño del artefacto inspeccionado. También puede analizarse la productividad de la revisión considerando los defectos identificados respecto al esfuerzo invertido.

No obstante, una cantidad elevada de defectos detectados no necesariamente significa que el proceso de requisitos sea de mejor calidad; puede indicar que la inspección ha sido efectiva, pero también que el documento inicial contenía numerosos problemas. Por ello, las métricas deben interpretarse conjuntamente y compararse entre revisiones o versiones del ERS.

Es necesario además considerar la composición del equipo de inspección, puesto que la incorporación de varios inspectores no garantiza que sus detecciones sean completamente independientes entre sí [14].

2.2.6. Caso de estudio o ejemplo aplicado

Como ejemplo de aplicación, considérese la validación de un requisito perteneciente al proyecto MundiPets:

RF-01: El sistema permitirá registrar una mascota asociándola con los datos correspondientes a su propietario.

Durante una revisión del requisito, el equipo podría identificar que la expresión “datos correspondientes” no especifica cuáles son obligatorios ni establece qué debe ocurrir si el propietario todavía no se encuentra registrado. Además, tampoco indica restricciones sobre la información de la mascota ni las condiciones que determinan que el registro se haya realizado correctamente.

Como resultado de la validación, el requisito podría descomponerse o complementarse con criterios específicos relacionados con los datos obligatorios, las validaciones de entrada y el comportamiento esperado ante situaciones excepcionales. Posteriormente, los stakeholders deberían confirmar que las modificaciones representan correctamente el proceso que desean implementar.

Este ejemplo evidencia que la validación no se limita a comprobar la gramática del requisito. Su objetivo es descubrir información ausente, ambigua, inconsistente o incorrecta antes de que el requisito sea utilizado como fundamento para el diseño y desarrollo. Además, cualquier modificación resultante deberá incorporarse mediante los mecanismos de gestión de requisitos tratados en la Sección 3.3, manteniendo su versión y trazabilidad.

2.3. Gestión de requisitos

La gestión de requisitos comprende el conjunto de actividades destinadas a mantener los requisitos identificados, organizados, actualizados y trazables durante el ciclo de vida del sistema. Su importancia aumenta conforme evoluciona el proyecto, debido a que los requisitos pueden modificarse como consecuencia de nuevas necesidades de los stakeholders, restricciones técnicas, cambios del entorno o decisiones tomadas durante el desarrollo.

Gestionar los requisitos implica, por tanto, mantener información sobre su estado, origen, prioridad, versión y relaciones con otros elementos del proyecto. La trazabilidad constituye uno de los principales mecanismos para conseguirlo, ya que permite establecer vínculos entre los requisitos y los artefactos que los originan o que posteriormente se producen a partir de ellos.

Mantener los requisitos actualizados y comprender su contexto resulta especialmente importante conforme los proyectos aumentan de tamaño y complejidad. La trazabilidad permite responder preguntas relacionadas con el origen de un requisito, los motivos de sus modificaciones y las personas involucradas en su definición [6].

2.3.1. Gestión de la configuración del ERS

La Especificación de Requisitos de Software (ERS) constituye uno de los principales artefactos documentales de la ingeniería de requisitos. Debido a que puede evolucionar durante el proyecto, resulta necesario gestionar su configuración para identificar las versiones existentes y mantener control sobre las modificaciones realizadas.

La gestión de configuración del ERS permite determinar qué versión se encuentra vigente, qué requisitos fueron modificados, cuándo se produjeron los cambios y cuáles son los elementos afectados. De esta manera, se evita que diferentes miembros del equipo trabajen con versiones incompatibles de la especificación y se conserva un historial de su evolución.

Este control adquiere especial importancia cuando los requisitos mantienen relaciones con otros artefactos. Un cambio realizado en el ERS puede repercutir posteriormente en elementos de diseño, código o pruebas, por lo que la configuración y la trazabilidad deben mantenerse coordinadas.

2.3.2. Línea base (baseline) y control de versiones

Una línea base o *baseline* representa una versión formalmente establecida de un conjunto de requisitos que sirve como referencia para continuar el desarrollo. Una vez establecida, sus elementos no deberían modificarse de manera informal; los cambios posteriores deben someterse al procedimiento de control definido por el proyecto.

La línea base no significa que los requisitos sean inmutables. Su propósito consiste en proporcionar un punto de referencia conocido frente al cual puedan identificarse y evaluar las modificaciones posteriores. Así, cuando se aprueba un cambio, se registra la modificación y se genera una nueva versión manteniendo el historial correspondiente.

El control de versiones complementa este mecanismo al permitir identificar la evolución del ERS y de sus requisitos. Como mínimo, debe ser posible conocer qué elemento cambió, cuál era su contenido anterior, cuál es su estado actual y qué modificación dio lugar a la nueva versión.

2.3.3. Atributos de los requisitos

Además de su descripción textual, los requisitos pueden incorporar atributos que proporcionan información adicional para facilitar su gestión. Estos atributos permiten identificar, clasificar, priorizar y controlar los requisitos durante su evolución.

Entre los atributos que pueden asociarse a un requisito se encuentran un identificador único, estado, prioridad, responsable, fuente u origen, versión, fecha de creación o modificación y criterios asociados a su verificación. La selección concreta de atributos dependerá de las necesidades del proyecto y de su proceso de gestión.

Por ejemplo, un requisito funcional podría registrarse de la siguiente manera:

Identificador: RF-01
Descripción: Registrar mascota.
Prioridad: Alta.
Fuente: Stakeholder.
Estado: Aprobado.
Versión: 1.1.

Esta información facilita posteriormente operaciones como localizar requisitos, identificar modificaciones, determinar responsabilidades y establecer relaciones de trazabilidad. Por ello, los atributos no sustituyen al contenido del requisito, sino que proporcionan los metadatos necesarios para administrarlo durante su ciclo de vida.

2.3.4. Trazabilidad: pre-traceability y post-traceability

La trazabilidad de requisitos es la capacidad de establecer y mantener relaciones entre un requisito y la información vinculada con su origen, evolución e implementación. Estas relaciones permiten comprender por qué existe determinado requisito y qué elementos podrían verse afectados si se modifica.

Se distingue entre *pre-requirements specification traceability* (pre-RS) y *post-requirements specification traceability* (post-RS). La primera establece relaciones entre los requisitos y sus fuentes anteriores a su incorporación en la especificación, como entrevistas con stakeholders, reuniones, documentos organizacionales o sistemas existentes [6]. Por ejemplo:

Entrevista con propietario → necesidad identificada → RF-01 Registrar mascota.

La post-traceability, por otra parte, establece relaciones entre los requisitos especificados y los artefactos

generados posteriormente durante el desarrollo, como elementos de diseño, código fuente y casos de prueba. Por ejemplo:

RF-01 Registrar mascota → módulo de registro → código correspondiente → CP-01 Prueba de registro.

Por tanto, ambas perspectivas permiten reconstruir el ciclo del requisito: la pre-traceability ayuda a comprender de dónde proviene, mientras que la post-traceability permite conocer cómo fue desarrollado y comprobado.

La investigación reciente continúa demostrando la importancia y, al mismo tiempo, las dificultades prácticas de mantener estos vínculos, señalando las barreras que dificultan la adopción de trazabilidad en proyectos reales y una menor atención histórica hacia la pre-trazabilidad en comparación con la trazabilidad posterior a la especificación [6, 15].

2.3.5. Matriz de trazabilidad

Una matriz de trazabilidad de requisitos constituye una representación estructurada de las relaciones existentes entre requisitos y otros elementos del proyecto. Su objetivo es facilitar la comprobación de cobertura y permitir identificar rápidamente qué artefactos están relacionados con determinado requisito.

Una representación simplificada para MundiPets podría ser la que se muestra en la Tabla 1.

Tabla 1: Ejemplo simplificado de matriz de trazabilidad para MundiPets.

ID	Requisito	Fuente	Diseño/Módulo	Caso de prueba
RF-01	Registrar mascota	Entrevista E01	Gestión de mascotas	CP-01
RF-02	Consultar mascota	Entrevista E02	Gestión de mascotas	CP-02
RF-03	Actualizar datos	Entrevista E02	Gestión de mascotas	CP-03

La matriz permite realizar tanto seguimiento hacia adelante como hacia atrás. Por ejemplo, partiendo de RF-01 puede localizarse el módulo y caso de prueba relacionados; en sentido contrario, partiendo de CP-01 puede identificarse qué requisito justifica la existencia de dicha prueba.

Esta información también resulta útil durante el análisis de cambios. Si RF-01 se modifica, los enlaces permiten identificar los artefactos potencialmente afectados y determinar cuáles deben revisarse.

No obstante, mantener trazabilidad tiene un costo organizacional. Un estudio empírico encontró que, aunque existen numerosas técnicas y herramientas de trazabilidad, su utilización práctica continúa enfrentándose a distintas barreras [15].

2.3.6. Proceso de cambio: RFC, análisis de impacto, CCB, implementación y cierre

Los requisitos pueden cambiar durante el desarrollo, pero sus modificaciones deben gestionarse de manera controlada cuando forman parte de una línea base. Un procedimiento estructurado evita que una modificación sea incorporada sin conocer previamente sus consecuencias [6].

El proceso puede comenzar mediante una *Request for Change* (RFC) o solicitud de cambio, en la cual se registra la modificación propuesta y su justificación. Posteriormente se realiza un análisis de impacto, considerando los requisitos relacionados y los artefactos que podrían verse afectados, además de factores como esfuerzo, costo, riesgos y planificación [5].

Con esta información, el Change Control Board (CCB) o la autoridad establecida por el proyecto puede evaluar la solicitud y decidir su aprobación, rechazo o aplazamiento. Si el cambio es aprobado, se procede a su implementación, actualizando el requisito y los elementos relacionados. Finalmente se realiza el cierre, verificando que las modificaciones hayan sido efectuadas correctamente, actualizando las versiones y conservando la trazabilidad del cambio.

De manera resumida, el flujo puede representarse como:

RFC → registro → análisis de impacto → evaluación del CCB → aprobación/rechazo → implementación → verificación → actualización de línea base → cierre.

La trazabilidad desempeña un papel fundamental durante este procedimiento porque permite determinar las posibles consecuencias de modificar un requisito. La literatura reciente señala precisamente que los enlaces de trazabilidad ayudan a comprender el contexto y evolución de los requisitos, incluyendo las razones que motivaron determinadas modificaciones [6].

2.3.7. Métricas de volatilidad de requisitos

La volatilidad de requisitos representa el grado en que los requisitos cambian durante un período determinado del proyecto. Su seguimiento permite identificar situaciones en las que se están produciendo modificaciones

frecuentes y analizar si estas pueden afectar la estabilidad del desarrollo.

Entre los elementos que pueden contabilizarse se encuentran los requisitos añadidos, modificados y eliminados durante un período. A partir de estos datos pueden establecerse indicadores relativos respecto al número total de requisitos existentes en la línea base.

Por ejemplo, para fines de seguimiento interno podría utilizarse una razón como:

$$V = \frac{R_a + R_m + R_e}{R_t} \times 100 \quad (1)$$

donde R_a representa los requisitos añadidos, R_m los modificados, R_e los eliminados y R_t el número de requisitos tomado como referencia para el período. Esta expresión debe entenderse como un indicador operativo de seguimiento y no como una fórmula universal establecida por ISO/IEC/IEEE 29148.

Si una línea base contiene 50 requisitos y durante un período se añaden 2, se modifican 3 y se elimina 1, el indicador sería:

$$V = \frac{2 + 3 + 1}{50} \times 100 = 12 \quad (2)$$

El resultado indica que durante ese período se produjo actividad de cambio equivalente al 12 % del conjunto tomado como referencia. Sin embargo, el porcentaje no debe interpretarse aisladamente como bueno o malo; debe analizarse junto con la etapa del proyecto, las causas de los cambios y sus impactos.

Un estudio que aborda específicamente la *Requirement Change Volatility* (RCV) señala que una planificación insuficiente frente a cambios destinados a responder a las expectativas de los stakeholders puede producir propagación de modificaciones y contribuir a problemas en el proyecto. Su investigación utiliza redes de requisitos y técnicas de aprendizaje automático para evaluar y predecir esta volatilidad [16].

De esta manera, las métricas de volatilidad complementan la gestión de cambios: mientras la RFC y el análisis de impacto permiten administrar una modificación concreta, las métricas permiten observar el comportamiento agregado de los cambios a lo largo del proyecto.

2.4. Herramientas CASE para IR

Las herramientas CASE (*Computer-Aided Software Engineering*) constituyen el soporte tecnológico que automatiza, total o parcialmente, las actividades de elicitación, especificación, verificación, validación y gestión de los requisitos a lo largo del ciclo de vida del software. Su adopción responde a la necesidad de reducir el esfuerzo manual asociado a la trazabilidad, el control de versiones y la comunicación entre los interesados, actividades que, gestionadas sin apoyo automatizado, incrementan la probabilidad de inconsistencias en el ERS [3].

2.4.1. Clasificación de herramientas CASE

Las herramientas CASE aplicadas a la ingeniería de requisitos (IR) se clasifican habitualmente según la etapa del proceso que automatizan y el alcance funcional que ofrecen. Un análisis reciente de 56 herramientas de IR agrupó sus características en siete perspectivas complementarias: gestión de proyectos, especificación, colaboración, personalización, interoperabilidad, metodología y soporte al usuario, evidenciando que ninguna herramienta cubre de manera uniforme la totalidad de estas dimensiones [17]. Desde una perspectiva funcional más orientada al mercado, otros estudios distinguen entre herramientas de *requirements gathering* (apoyadas en técnicas visuales, diagramas de contexto, descomposición funcional e historias de usuario) y herramientas de *requirements management* propiamente dichas, orientadas al control de cambios, la trazabilidad y la generación de reportes [18]. Esta doble clasificación por etapa del proceso y por alcance funcional resulta coherente con el proceso de validación y gestión descrito en el estándar ISO/IEC/IEEE 29148:2018 y con los procesos de control de cambios de la ISO/IEC/IEEE 12207:2017 [3, 5].

2.4.2. Funcionalidades esperables en gestión de requisitos

Con independencia de la clasificación adoptada, la literatura converge en un conjunto de funcionalidades mínimas esperables en una herramienta CASE orientada a la gestión de requisitos. Ozkaya et al. [17] identifican, entre las más recurrentes, la especificación estructurada de requisitos, el versionado, la trazabilidad bidireccional, el soporte a la colaboración distribuida entre analistas y stakeholders, y la interoperabilidad con otras herramientas del ciclo de vida (gestión de pruebas, control de código y gestión de proyectos). En la misma línea, la comparación cualitativa de diez herramientas comerciales de gestión de requisitos realizada por Nadeem et al. [18] evalúa explícitamente diez parámetros –seguimiento de errores, gestión jerárquica de requisitos, diseño

visual, gestión de riesgos, gestión de pruebas, verificación de requisitos, análisis de calidad, versionado, gestión de *releases* y generación de reportes–, concluyendo que ninguna herramienta analizada obtiene un desempeño uniformemente alto en todos ellos, lo que refuerza la necesidad de un criterio de selección alineado con las prioridades específicas del proyecto.

2.4.3. Criterios de selección de una herramienta CASE

La selección de una herramienta CASE no es un problema trivial: involucra criterios técnicos, organizacionales y económicos que deben ponderarse según el contexto del proyecto. Bjarnason et al. [19] proponen un modelo de selección de software a gran escala construido a partir de revisiones rápidas e iterativas, que combina una lista de criterios de evaluación (funcionales, de usabilidad, de soporte del proveedor y de costo total de propiedad) con un proceso estructurado para identificar, valorar y filtrar alternativas antes de la adopción definitiva. Este enfoque es aplicable directamente a la selección de herramientas de gestión de requisitos, dado que ambos dominios comparten la dificultad de comparar productos con conjuntos de características parcialmente solapados y una oferta comercial heterogénea. Entre los criterios más citados en la literatura destacan la facilidad de integración con el resto del ecosistema de herramientas del proyecto, la curva de aprendizaje, el soporte a estándares de la industria y la escalabilidad para equipos distribuidos [17, 19].

2.4.4. Limitaciones y costos

La adopción de herramientas CASE no está exenta de limitaciones. En primer lugar, el costo total de propiedad –licenciamiento, capacitación del equipo y mantenimiento– constituye una barrera relevante, particularmente para organizaciones pequeñas o proyectos con presupuestos ajustados [19]. En segundo lugar, el análisis comparativo de Nadeem et al. [18] evidencia que las herramientas orientadas a funcionalidades avanzadas (gestión de riesgos, análisis de calidad de requisitos) suelen sacrificar simplicidad de uso, lo que puede traducirse en una curva de adopción prolongada. Finalmente, Ozkaya et al. [17] señalan que la interoperabilidad entre herramientas de distintos proveedores continúa siendo una limitación estructural del mercado, lo que obliga frecuentemente a las organizaciones a asumir integraciones a medida o a tolerar silos de información entre la especificación de requisitos y otras disciplinas del ciclo de vida (pruebas, arquitectura, gestión de proyectos).

2.5. IR en procesos ágiles

La incorporación de prácticas ágiles ha transformado sustancialmente la forma en que se elicitán, documentan y gestionan los requisitos, desplazando el énfasis desde la especificación exhaustiva y previa hacia una construcción incremental y colaborativa del entendimiento del producto. Esta sección revisa los artefactos, prácticas y mecanismos de trazabilidad propios de la IR ágil, y los contrasta con el enfoque documental descrito en 3.1–3.3.

2.5.1. Product backlog, historias de usuario y criterios de aceptación

El *product backlog* constituye el artefacto central de la gestión ágil de requisitos: una lista ordenada y viva de necesidades del producto que se refina de manera continua. Su unidad de trabajo predominante es la historia de usuario, cuyo origen se remonta a la programación extrema y que se popularizó como mecanismo para capturar requisitos desde la perspectiva del usuario final, priorizando la conversación sobre la documentación exhaustiva [20]. No obstante, la sola redacción de una historia de usuario no garantiza su calidad: el estudio de caso de Kuhail y Lauesen [21], que evaluó las respuestas de ocho profesionales de TI frente a un proyecto real utilizando los criterios de calidad de la IEEE 830 (completitud, corrección, verificabilidad y trazabilidad), encontró que las historias de usuario producidas cubrieron apenas el 33 % de las necesidades identificadas en el informe de análisis original, lo que evidencia el riesgo de pérdida de información al migrar de una especificación documental a un formato ágil sin mecanismos de control adicionales. Por ello, Ferreira et al. [20] proponen enriquecer las historias de usuario con criterios de aceptación explícitos, épicas y temas que permitan preservar la trazabilidad conceptual entre el requisito de alto nivel y su implementación incremental, en línea con los atributos de calidad definidos en la ISO/IEC/IEEE 29148:2018 [3].

2.5.2. Grooming y Definition of Ready / Definition of Done

El refinamiento del backlog (*backlog grooming* o *refinement*) es la actividad continua mediante la cual el equipo desglosa, estima y clarifica los elementos del backlog hasta que cumplen un nivel de detalle suficiente para ser abordados en un sprint. La Guía de Scrum no define formalmente una *Definition of Ready* (DoR), pero establece que los elementos del backlog deben alcanzar un grado de transparencia y comprensión que permita completarlos dentro de un sprint [22]. La *Definition of Done* (DoD), en cambio, sí forma parte del marco oficial de Scrum y constituye un compromiso formal y compartido por el equipo sobre lo que significa que un incremento esté terminado, siendo responsabilidad primaria del equipo de desarrollo mantener su cumplimiento [22]. Tomaselli et al. [23], en su revisión sistemática sobre métodos de gestión de requisitos en desarrollo ágil, destacan que

la ausencia de criterios de entrada y salida explícitos –es decir, de un DoR y un DoD bien definidos– es una de las causas recurrentes de retrabajo y de defectos de calidad en los incrementos entregados, y recomiendan su integración temprana como práctica de gestión de requisitos dentro del marco de calidad ágil (*Agile Quality Framework*).

2.5.3. BDD y formato Gherkin

El desarrollo guiado por comportamiento (*Behavior-Driven Development*, BDD) extiende la lógica de las historias de usuario hacia especificaciones ejecutables, escritas en lenguaje natural estructurado mediante el formato Gherkin (*Given-When-Then*), que permite a analistas, desarrolladores y evaluadores compartir un mismo artefacto como documentación, criterio de aceptación y base para pruebas automatizadas. La revisión sistemática de Abushama et al. [24] sobre el efecto del desarrollo guiado por pruebas (TDD) y del BDD en los factores de éxito de un proyecto concluye que ambas prácticas inciden positivamente en la satisfacción del cliente y en la reducción de defectos, atribuyendo al BDD un valor adicional en la mejora de la comunicación entre perfiles técnicos y no técnicos gracias a la legibilidad de sus escenarios. Esta característica convierte al formato Gherkin en un mecanismo natural de validación de requisitos en entornos ágiles, ya que cada escenario actúa simultáneamente como criterio de aceptación verificable y como evidencia de trazabilidad entre la necesidad del negocio y el comportamiento implementado del sistema.

2.5.4. Trazabilidad en entornos ágiles

La trazabilidad de requisitos –la capacidad de seguir un requisito desde su origen hasta su implementación y verificación– presenta desafíos particulares en contextos ágiles, donde la reducción deliberada de la documentación formal puede debilitar los enlaces explícitos entre artefactos. El estudio de mapeo de Charalampidou et al. [25] sobre trazabilidad de software identifica que, si bien la mayoría de los métodos de trazabilidad fueron concebidos originalmente para procesos secuenciales y documentales, existe una tendencia creciente hacia enfoques ligeros y semiautomatizados basados en recuperación de información y en la vinculación de commits, historias de usuario y casos de prueba que se adaptan mejor a la cadencia iterativa del desarrollo ágil. En la misma dirección, Loh Mun Keong y Che Embi [26] advierten que la documentación de requisitos no funcionales es una de las prácticas más frecuentemente descuidadas en equipos ágiles, lo que compromete la trazabilidad de atributos de calidad como los definidos en la ISO/IEC 25010:2023 [7] y exige mecanismos complementarios (etiquetado de historias, matrices ligeras o herramientas CASE integradas) para no perder dicha trazabilidad frente a la mayor velocidad de entrega.

2.5.5. IR documental frente a IR ágil: ¿cuándo conviene cada enfoque?

La disyuntiva entre un enfoque documental –propio de los estándares revisados en 3.1 a 3.3, con énfasis en el ERS, la línea base y la matriz de trazabilidad formal– y un enfoque ágil –centrado en el backlog vivo, las historias de usuario y la trazabilidad ligera– no debe entenderse como una elección excluyente, sino como una decisión de diseño de proceso condicionada por el contexto del proyecto. La revisión de Tomaselli et al. [23] concluye que los proyectos con alta variabilidad de requisitos, plazos cortos y participación constante del cliente se benefician de prácticas ágiles de gestión de requisitos, mientras que proyectos regulados, de misión crítica o con requisitos contractuales estrictos –por ejemplo, aquellos sujetos a normas como la ISO/IEC/IEEE 29148:2018 o a certificaciones sectoriales– requieren preservar un nivel de documentación formal y trazabilidad verificable que los formatos puramente ágiles no garantizan por defecto [3]. En la práctica, numerosas organizaciones adoptan esquemas híbridos: mantienen una especificación de alto nivel documentada y una línea base contractual, mientras gestionan el detalle operativo mediante backlog, historias de usuario y escenarios Gherkin, combinando así la trazabilidad formal exigida por el marco normativo con la agilidad necesaria para responder a los cambios propios del desarrollo iterativo [23, 26].

3. Inspección formal del ERS (método Fagan)

3.1. Planificación y roles

3.2. Preparación individual

3.3. Acta de la reunión de inspección

3.4. Métricas de inspección

4. Correcciones aplicadas

5. Gestión del cambio y Change Control Board (CCB)

- 5.1. RFC-01 – Cambio de alcance (nuevo requisito)**
- 5.2. RFC-02 – Cambio de calidad (modificación de un RNF)**
- 5.3. RFC-03 – Cambio de restricción o normativa**
- 5.4. Acta del CCB**
- 5.5. Versión resultante del ERS**

6. Trazabilidad en herramienta CASE

- 6.1. Estructura del tablero**
- 6.2. Campos personalizados**
- 6.3. Matriz de trazabilidad**
- 6.4. Evidencias del tablero**

7. Línea base del ERS con Git

7.1. Repositorio y estructura

7.2. Commits semánticos

7.3. Tag anotado de línea base

7.4. CHANGELOG.md

7.5. Historial de commits (evidencia)

8. Comparativa de herramientas CASE

9. IR tradicional frente a IR ágil

10. Conclusiones críticas

11. Distribución del trabajo

12. Referencias

- [1] L. Montgomery, D. Fucci, A. Bouraffa, L. Scholz y W. Maalej, “Empirical research on requirements quality: a systematic mapping study,” *Requirements Engineering*, vol. 27, págs. 183-209, 2 jun. de 2022, ISSN: 0947-3602. DOI: 10.1007/s00766-021-00367-z
- [2] M. A. Umar y K. Lano, “Advances in automated support for requirements engineering: a systematic literature review,” *Requirements Engineering*, vol. 29, págs. 177-207, 2 jun. de 2024, ISSN: 0947-3602. DOI: 10.1007/s00766-023-00411-0
- [3] ISO/IEC/IEEE, “ISO/IEC/IEEE 29148:2018 – Systems and Software Engineering – Life Cycle Processes – Requirements Engineering,” International Organization for Standardization, inf. téc., 2018.
- [4] ISO/IEC/IEEE, *15288:2023 — System life cycle processes*, 2023.
- [5] ISO/IEC/IEEE, “ISO/IEC/IEEE 12207:2017 – Systems and Software Engineering – Software Life Cycle Processes,” International Organization for Standardization, inf. téc., 2017.
- [6] J. Mucha, A. Kaufmann y D. Riehle, “A systematic literature review of pre-requirements specification traceability,” *Requirements Engineering*, vol. 29, págs. 119-141, 2 jun. de 2024, ISSN: 0947-3602. DOI: 10.1007/s00766-023-00412-z
- [7] ISO/IEC, “ISO/IEC 25010:2023 – Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation (SQuaRE) – Product Quality Model,” International Organization for Standardization, inf. téc., 2023.
- [8] H. Washizaki, *SWEBOK Guide v4.0*, H. Washizaki, ed. IEEE Computer Society, 2024. dirección: <https://www.computer.org/education/bodies-of-knowledge/software-engineering>
- [9] H. Washizaki y J. Olszewska, “Guide to the Software Engineering Body of Knowledge v4.0,” vol. 4.0, pág. 413, 2024.
- [10] K. Pohl, *Requirements engineering : fundamentals, principles, and techniques*. Springer, 2025, pág. 814, ISBN: 9783662692042.
- [11] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK Guide) – Seventh Edition*. Project Management Institute, 2021.
- [12] W. Abdeen, X. Chen y M. Unterkalmsteiner, “An approach for performance requirements verification and test environments generation,” *Requirements Engineering*, abr. de 2022, ISSN: 0947-3602. DOI: 10.1007/s00766-022-00379-3
- [13] U.-e. Habiba, M. Haug, J. Bogner y S. Wagner, “How mature is requirements engineering for AI-based systems? A systematic mapping study on practices, challenges, and future research directions,” *Requirements Engineering*, vol. 29, págs. 567-600, 4 dic. de 2024, ISSN: 0947-3602. DOI: 10.1007/s00766-024-00432-3
- [14] B. Li y C. Smidts, “A Zone-Based Model for Analysis of Dependent Failures in Requirements Inspection,” *IEEE Transactions on Software Engineering*, vol. 49, págs. 3581-3598, 6 jun. de 2023, ISSN: 0098-5589. DOI: 10.1109/TSE.2023.3266157
- [15] M. Ruiz, J. Y. Hu y F. Dalpiaz, “Why don’t we trace? A study on the barriers to software traceability in practice,” *Requirements Engineering*, vol. 28, págs. 619-637, 4 dic. de 2023, ISSN: 0947-3602. DOI: 10.1007/s00766-023-00408-9
- [16] P. H. Hein, E. Kames, C. Chen y B. Morkos, “Employing machine learning techniques to assess requirement change volatility,” *Research in Engineering Design*, vol. 32, págs. 245-269, 2 abr. de 2021, ISSN: 0934-9839. DOI: 10.1007/s00163-020-00353-6
- [17] M. Ozkaya, G. Kardas y M. A. Kose, “An Analysis of the Features of Requirements Engineering Tools,” *Systems*, vol. 11, n.º 12, pág. 576, 2023. DOI: 10.3390/systems11120576 dirección: <https://doi.org/10.3390/systems11120576>
- [18] M. A. Nadeem, S. U.-J. Lee y M. U. Younus, “A Comparison of Recent Requirements Gathering and Management Tools in Requirements Engineering for IoT-Enabled Sustainable Cities,” *Sustainability*, vol. 14, n.º 4, pág. 2427, 2022. DOI: 10.3390/su14042427 dirección: <https://doi.org/10.3390/su14042427>
- [19] E. Bjarnason, K. Sharma y P. Runeson, “Software Selection in Large-Scale Software Engineering: A Model and Criteria Based on Interactive Rapid Reviews,” *Empirical Software Engineering*, vol. 28, n.º 2, pág. 51, 2023. DOI: 10.1007/s10664-023-10288-w dirección: <https://doi.org/10.1007/s10664-023-10288-w>
- [20] A. Ferreira, A. R. Da Silva y A. C. R. Paiva, “Towards the Art of Writing Agile Requirements with User Stories, Acceptance Criteria, and Related Constructs,” en *Proceedings of the 17th International Conference on Evaluation of Novel Approaches to Software Engineering (ENASE 2022)*, 2022, págs. 477-484. DOI: 10.5220/0011082000003176 dirección: <https://doi.org/10.5220/0011082000003176>

- [21] M. A. Kuhail y S. Lauesen, “User Story Quality in Practice: A Case Study,” *Software*, vol. 1, n.º 3, págs. 223-243, 2022. DOI: 10.3390/software1030010 dirección: <https://doi.org/10.3390/software1030010>
- [22] K. Schwaber y J. Sutherland, *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*, 2020. dirección: <https://scrumguides.org/>
- [23] G. P. Tomaselli, N. S. Pinto y C. Acuña, “Requirements Management Methods and Practices for Improving the Quality of Agile Software Development Processes: A Review of the Literature,” *Requirements Engineering*, vol. 30, págs. 371-398, 2025. DOI: 10.1007/s00766-025-00448-3 dirección: <https://doi.org/10.1007/s00766-025-00448-3>
- [24] H. M. Abushama, H. A. Alassam y F. A. Elhaj, “The Effect of Test-Driven Development and Behavior-Driven Development on Project Success Factors: A Systematic Literature Review Based Study,” en *2020 International Conference on Computer, Control, Electrical, and Electronics Engineering (ICCCEEE)*, 2021, págs. 1-9. DOI: 10.1109/ICCCEEE49695.2021.9429593 dirección: <https://doi.org/10.1109/ICCCEEE49695.2021.9429593>
- [25] S. Charalampidou, A. Ampatzoglou, E. Karountzos y P. Avgeriou, “Empirical Studies on Software Traceability: A Mapping Study,” *Journal of Software: Evolution and Process*, vol. 33, n.º 2, e2294, 2021. DOI: 10.1002/smr.2294 dirección: <https://doi.org/10.1002/smr.2294>
- [26] S. Loh Mun Keong y Z. Che Embi, “A Systematic Review on Non-Functional Requirements Documentation in Agile Methodology,” *Journal of Informatics and Web Engineering*, vol. 1, n.º 2, págs. 19-29, 2022. DOI: 10.33093/jiwe.2022.1.2.2 dirección: <https://doi.org/10.33093/jiwe.2022.1.2.2>

Anexo A — Lista de verificación de inspección (checklists individuales)

Anexo B — Registro de defectos

Anexo C — Formularios RFC y acta del CCB

Anexo D — Exportación CSV del backlog

Anexo E — Capturas del tablero CASE y del repositorio Git

Anexo F — Referencias IEEE y declaración de uso de Inteligencia Artificial

Referencias

- [1] L. Montgomery, D. Fucci, A. Bouraffa, L. Scholz y W. Maalej, “Empirical research on requirements quality: a systematic mapping study,” *Requirements Engineering*, vol. 27, págs. 183-209, 2 jun. de 2022, ISSN: 0947-3602. DOI: 10.1007/s00766-021-00367-z
- [2] M. A. Umar y K. Lano, “Advances in automated support for requirements engineering: a systematic literature review,” *Requirements Engineering*, vol. 29, págs. 177-207, 2 jun. de 2024, ISSN: 0947-3602. DOI: 10.1007/s00766-023-00411-0
- [3] ISO/IEC/IEEE, “ISO/IEC/IEEE 29148:2018 – Systems and Software Engineering – Life Cycle Processes – Requirements Engineering,” International Organization for Standardization, inf. téc., 2018.
- [4] ISO/IEC/IEEE, *15288:2023 — System life cycle processes*, 2023.
- [5] ISO/IEC/IEEE, “ISO/IEC/IEEE 12207:2017 – Systems and Software Engineering – Software Life Cycle Processes,” International Organization for Standardization, inf. téc., 2017.
- [6] J. Mucha, A. Kaufmann y D. Riehle, “A systematic literature review of pre-requirements specification traceability,” *Requirements Engineering*, vol. 29, págs. 119-141, 2 jun. de 2024, ISSN: 0947-3602. DOI: 10.1007/s00766-023-00412-z
- [7] ISO/IEC, “ISO/IEC 25010:2023 – Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation (SQuaRE) – Product Quality Model,” International Organization for Standardization, inf. téc., 2023.
- [8] H. Washizaki, *SWEBOK Guide v4.0*, H. Washizaki, ed. IEEE Computer Society, 2024. dirección: <https://www.computer.org/education/bodies-of-knowledge/software-engineering>
- [9] H. Washizaki y J. Olszewska, “Guide to the Software Engineering Body of Knowledge v4.0,” vol. 4.0, pág. 413, 2024.
- [10] K. Pohl, *Requirements engineering : fundamentals, principles, and techniques*. Springer, 2025, pág. 814, ISBN: 9783662692042.
- [11] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK Guide) – Seventh Edition*. Project Management Institute, 2021.
- [12] W. Abdeen, X. Chen y M. Unterkalmsteiner, “An approach for performance requirements verification and test environments generation,” *Requirements Engineering*, abr. de 2022, ISSN: 0947-3602. DOI: 10.1007/s00766-022-00379-3
- [13] U.-e. Habiba, M. Haug, J. Bogner y S. Wagner, “How mature is requirements engineering for AI-based systems? A systematic mapping study on practices, challenges, and future research directions,” *Requirements Engineering*, vol. 29, págs. 567-600, 4 dic. de 2024, ISSN: 0947-3602. DOI: 10.1007/s00766-024-00432-3
- [14] B. Li y C. Smidts, “A Zone-Based Model for Analysis of Dependent Failures in Requirements Inspection,” *IEEE Transactions on Software Engineering*, vol. 49, págs. 3581-3598, 6 jun. de 2023, ISSN: 0098-5589. DOI: 10.1109/TSE.2023.3266157
- [15] M. Ruiz, J. Y. Hu y F. Dalpiaz, “Why don’t we trace? A study on the barriers to software traceability in practice,” *Requirements Engineering*, vol. 28, págs. 619-637, 4 dic. de 2023, ISSN: 0947-3602. DOI: 10.1007/s00766-023-00408-9
- [16] P. H. Hein, E. Kames, C. Chen y B. Morkos, “Employing machine learning techniques to assess requirement change volatility,” *Research in Engineering Design*, vol. 32, págs. 245-269, 2 abr. de 2021, ISSN: 0934-9839. DOI: 10.1007/s00163-020-00353-6
- [17] M. Ozkaya, G. Kardas y M. A. Kose, “An Analysis of the Features of Requirements Engineering Tools,” *Systems*, vol. 11, n.º 12, pág. 576, 2023. DOI: 10.3390/systems11120576 dirección: <https://doi.org/10.3390/systems11120576>
- [18] M. A. Nadeem, S. U.-J. Lee y M. U. Younus, “A Comparison of Recent Requirements Gathering and Management Tools in Requirements Engineering for IoT-Enabled Sustainable Cities,” *Sustainability*, vol. 14, n.º 4, pág. 2427, 2022. DOI: 10.3390/su14042427 dirección: <https://doi.org/10.3390/su14042427>
- [19] E. Bjarnason, K. Sharma y P. Runeson, “Software Selection in Large-Scale Software Engineering: A Model and Criteria Based on Interactive Rapid Reviews,” *Empirical Software Engineering*, vol. 28, n.º 2, pág. 51, 2023. DOI: 10.1007/s10664-023-10288-w dirección: <https://doi.org/10.1007/s10664-023-10288-w>
- [20] A. Ferreira, A. R. Da Silva y A. C. R. Paiva, “Towards the Art of Writing Agile Requirements with User Stories, Acceptance Criteria, and Related Constructs,” en *Proceedings of the 17th International Conference*

- on *Evaluation of Novel Approaches to Software Engineering (ENASE 2022)*, 2022, págs. 477-484. DOI: 10.5220/0011082000003176 dirección: <https://doi.org/10.5220/0011082000003176>
- [21] M. A. Kuhail y S. Lauesen, “User Story Quality in Practice: A Case Study,” *Software*, vol. 1, n.º 3, págs. 223-243, 2022. DOI: 10.3390/software1030010 dirección: <https://doi.org/10.3390/software1030010>
- [22] K. Schwaber y J. Sutherland, *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*, 2020. dirección: <https://scrumguides.org/>
- [23] G. P. Tomaselli, N. S. Pinto y C. Acuña, “Requirements Management Methods and Practices for Improving the Quality of Agile Software Development Processes: A Review of the Literature,” *Requirements Engineering*, vol. 30, págs. 371-398, 2025. DOI: 10.1007/s00766-025-00448-3 dirección: <https://doi.org/10.1007/s00766-025-00448-3>
- [24] H. M. Abushama, H. A. Alassam y F. A. Elhaj, “The Effect of Test-Driven Development and Behavior-Driven Development on Project Success Factors: A Systematic Literature Review Based Study,” en *2020 International Conference on Computer, Control, Electrical, and Electronics Engineering (ICCCEEE)*, 2021, págs. 1-9. DOI: 10.1109/ICCCEEE49695.2021.9429593 dirección: <https://doi.org/10.1109/ICCCEEE49695.2021.9429593>
- [25] S. Charalampidou, A. Ampatzoglou, E. Karountzos y P. Avgeriou, “Empirical Studies on Software Traceability: A Mapping Study,” *Journal of Software: Evolution and Process*, vol. 33, n.º 2, e2294, 2021. DOI: 10.1002/smr.2294 dirección: <https://doi.org/10.1002/smr.2294>
- [26] S. Loh Mun Keong y Z. Che Embi, “A Systematic Review on Non-Functional Requirements Documentation in Agile Methodology,” *Journal of Informatics and Web Engineering*, vol. 1, n.º 2, págs. 19-29, 2022. DOI: 10.33093/jiwe.2022.1.2.2 dirección: <https://doi.org/10.33093/jiwe.2022.1.2.2>

Uso de la IA